ST JOSEPH'S SCHOOL (CBSE)

Manavalakurichi, Kanyakumari District, Tamil Nadu

COMPUTER SCIENCE (083) PROJECT REPORT

Academic Year 2026-27

KADAL KANAKKU

Fish Catch, Auction & Sea-Safety Manager
for the Manavalakurichi Fish Landing Centre

(Python – MySQL Database Connectivity Project)

Name	Ryan Editraj
Class & Section	XII A
Roll No.	14
CBSE Board Roll No.	_____
Registration No.	_____
Board	Central Board of Secondary Education (CBSE)
Subject Teacher	_____

CERTIFICATE

This is to certify that Ryan Editraj of Class XII A, Roll No. 14, St Joseph's School (CBSE), Manavalakurichi, has successfully completed the Computer Science (083) project titled "KADAL KANAKKU – Fish Catch, Auction & Sea-Safety Manager" under my guidance during the academic year 2026-27, in partial fulfilment of the practical examination requirements of the Central Board of Secondary Education (CBSE).

Signature of Subject Teacher Name: _____	Signature of External Examiner Name: _____
Signature of Principal	School Seal

Date: _____

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my Computer Science teacher, _____, for her valuable guidance and encouragement throughout this project, and for advising me to choose an original topic. I thank our Principal and the management of St Joseph's School, Manavalakurichi, for providing the computer laboratory facilities.

I am thankful to the fishermen and fish merchants of Manavalakurichi, whose daily work at the beach landing centre inspired this project and helped me understand how catches are auctioned and how boats go to sea. Finally, I thank my parents and friends for their constant support.

Ryan Editraj
Class XII A

Contents

1. Synopsis / Introduction

2. Objectives

3. Hardware and Software Requirements

4. Proposed System and System Design

5. Database Design

6. Module Description

7. Source Code

8. Sample Outputs

9. Limitations and Future Scope

10. Bibliography

1. Synopsis / Introduction

Manavalakurichi is a coastal village in Kanyakumari district where most families depend on sea fishing. Every morning boats such as kattumarams, vallams, fibre boats and trawlers return to the beach, the catch is weighed and auctioned to exporters, wholesalers and local vendors. Today these records are kept in notebooks. It is difficult to find out how much a boat earned in a week, which species fetched the best price, or – most importantly – which boats and how many fishermen are still at sea when the weather turns rough.

KADAL KANAKKU ("sea accounts" in Tamil) is a menu-driven Python program with a MySQL database that solves this problem. It stores boats, fishermen, sea trips, catches and buyers in five related tables. It records each auction sale, applies a simple sea-safety rule before a boat is logged out, shows a live 'boats at sea' board, and produces reports such as daily revenue, species-wise totals, top-earning boats and buyer spending. A report can also be exported to a CSV file that opens in Excel.

The project uses the Class XI and XII syllabus concepts: functions, exception handling, CSV file handling, lists and tuples, SQL DDL/DML, keys and constraints, joins, aggregate functions, GROUP BY / HAVING, ORDER BY, LIKE, BETWEEN, IS NULL and Python–MySQL connectivity.

2. Objectives

- Maintain records of boats, fishermen and buyers of the landing centre.
- Log every sea trip with weather and crew count, and refuse unsafe trips (small boats in rough sea).
- Show at any time which boats and how many people are still at sea.
- Record catches and the final auction rate and buyer for each lot.
- Generate reports: daily revenue, species-wise totals, top boats, buyer spending, revenue between two dates.
- Export a report to a CSV file.
- Validate all user input and handle errors without the program crashing.

3. Hardware and Software Requirements

Hardware	**Minimum**
Processor	Intel Core i3 / equivalent or above
RAM	4 GB
Hard disk	1 GB free space
Others	Keyboard, monitor, printer (for report)

Software	**Details**
Operating System	Windows 10/11 or Linux

Front end	Python 3.10 or above (IDLE)
Back end	MySQL Server 8.0
Python module	mysql-connector-python (pip install mysql-connector-python)
Built-in modules	csv, datetime
Tested on	Python 3, MariaDB 11.4 (MySQL compatible) server

4. Proposed System and System Design

The program is divided into five sub-menus called from a main menu. Each menu option calls one user-defined function. Common tasks (reading a valid number, date or choice, checking whether a record exists, printing a table, running a query with commit/rollback) are written once as helper functions and reused.

```
MAIN MENU
 |-- 1. Boats & Fishermen : add/view boats, add fisherman, view crew (join),
 |                          search (LIKE), change owner (UPDATE), delete
 |-- 2. Trips & Sea Safety: start trip (safety check), mark returned,
 |                          boats at sea, all trips
 |-- 3. Catch & Auction   : record catch, unsold catches, auction a catch,
 |                          view all sales (4-table join), delete catch
 |-- 4. Buyers            : add buyer, view buyers
 |-- 5. Reports           : daily revenue, species totals, top 5 boats,
 |                          buyer spending (HAVING), revenue between dates,
 |                          export to CSV
 |-- 0. Exit (connection closed in finally block)
```

Work flow: Boat registered → trip started (status AT SEA) → trip marked RETURNED → catch recorded (buyer NULL = unsold) → auction updates buyer and rate → reports read the sold catches.

5. Database Design

Database name: kadal_kanakku. Five tables are related using primary and foreign keys:

```
   BOATS (boat_id PK) 1 ----< FISHERMEN (fisher_id PK, boat_id FK)
     |
     1
     ^
   TRIPS (trip_id PK, boat_id FK) 1 ----< CATCHES (catch_id PK, trip_id FK, buyer_id FK) >----
1 BUYERS (buyer_id PK)
   ( 1 ----< means one-to-many )
```

- A boat has many fishermen and makes many trips (one-to-many).
- A trip brings back many catch lots; each lot is sold to one buyer; a buyer buys many lots.
- boats.reg_no is UNIQUE (alternate key); fishermen.phone is UNIQUE.
- In catches, buyer_id and rate_per_kg are NULL until the lot is auctioned.

Table structures (output of DESC from the actual database)

```
mysql> DESC boats;
+------------+-------------+------+-----+---------+-------+
| Field      | Type        | Null | Key | Default | Extra |
```

```
+-------------+-------------+------+-----+---------+-------+
| boat_id     | int(11)     | NO   | PRI | NULL    |       |
| boat_name   | varchar(30) | NO   |     | NULL    |       |
| reg_no      | varchar(20) | NO   | UNI | NULL    |       |
| boat_type   | varchar(15) | NO   |     | NULL    |       |
| owner_name  | varchar(30) | NO   |     | NULL    |       |
| capacity_kg | int(11)     | YES  |     | NULL    |       |
+-------------+-------------+------+-----+---------+-------+
```

```
mysql> DESC fishermen;
+-----------+-------------+------+-----+---------+-------+
| Field     | Type        | Null | Key | Default | Extra |
+-----------+-------------+------+-----+---------+-------+
| fisher_id | int(11)     | NO   | PRI | NULL    |       |
| fname     | varchar(30) | NO   |     | NULL    |       |
| phone     | char(10)    | YES  | UNI | NULL    |       |
| village   | varchar(25) | YES  |     | NULL    |       |
| boat_id   | int(11)     | YES  | MUL | NULL    |       |
+-----------+-------------+------+-----+---------+-------+
```

```
mysql> DESC trips;
+------------+-------------+------+-----+---------+-------+
| Field      | Type        | Null | Key | Default | Extra |
+------------+-------------+------+-----+---------+-------+
| trip_id    | int(11)     | NO   | PRI | NULL    |       |
| boat_id    | int(11)     | NO   | MUL | NULL    |       |
| trip_date  | date        | NO   |     | NULL    |       |
| crew_count | int(11)     | YES  |     | NULL    |       |
| weather    | varchar(10) | YES  |     | NULL    |       |
| status     | varchar(10) | YES  |     | AT SEA  |       |
+------------+-------------+------+-----+---------+-------+
```

```
mysql> DESC buyers;
+----------+-------------+------+-----+---------+-------+
| Field    | Type        | Null | Key | Default | Extra |
+----------+-------------+------+-----+---------+-------+
| buyer_id | int(11)     | NO   | PRI | NULL    |       |
| bname    | varchar(30) | NO   |     | NULL    |       |
| btype    | varchar(15) | YES  |     | NULL    |       |
| town     | varchar(20) | YES  |     | NULL    |       |
| phone    | char(10)    | YES  |     | NULL    |       |
+----------+-------------+------+-----+---------+-------+
```

```
mysql> DESC catches;
+-------------+-------------+------+-----+---------+-------+
| Field       | Type        | Null | Key | Default | Extra |
+-------------+-------------+------+-----+---------+-------+
| catch_id    | int(11)     | NO   | PRI | NULL    |       |
| trip_id     | int(11)     | NO   | MUL | NULL    |       |
| species     | varchar(20) | NO   |     | NULL    |       |
| weight_kg   | float       | NO   |     | NULL    |       |
| buyer_id    | int(11)     | YES  | MUL | NULL    |       |
| rate_per_kg | float       | YES  |     | NULL    |       |
+-------------+-------------+------+-----+---------+-------+
```

Sample data size

Table	Degree (columns)	Cardinality (sample rows)
boats	6	12
fishermen	5	15
trips	6	15
buyers	5	10
catches	6	20

6. Module Description

Function(s)	Purpose
connect()	Connects to MySQL, creates global cursor
read_int / read_float / read_text / read_date / read_choice	Input validation using loops and try-except
exists()	Checks whether a record with a given key exists (COUNT)
show()	Prints query results as a formatted table
run()	Executes INSERT/UPDATE/DELETE with commit, rollback on error, returns rowcount
add_boat, view_boats, add_fisherman, view_crew, search_fisherman, update_owner, delete_fisherman	Boats & fishermen CRUD, JOIN and LIKE search
start_trip, mark_returned, boats_at_sea, view_trips	Trip log and sea-safety rules
record_catch, unsold_catches, auction_catch, view_sales, delete_catch	Catch and auction (IS NULL, 4-table join)
add_buyer, view_buyers	Buyer records
daily_report, species_report, top_boats, buyer_report, period_report	Reports using SUM, COUNT, AVG, MAX, GROUP BY, HAVING, ORDER BY, BETWEEN, LIMIT
export_csv	Writes species report with csv.writer and reads it back with csv.reader
main()	Main menu loop, try-finally closes connection

7. Source Code

7.1 schema.sql

```
-- ============================================================
--  KADAL KANAKKU : Fish Catch, Auction & Sea-Safety Manager
--  Manavalakurichi Fish Landing Centre
--  CBSE Class XII Computer Science (083) Project 2026-27
--  Ryan Editraj, XII A
-- ============================================================
DROP DATABASE IF EXISTS kadal_kanakku;
CREATE DATABASE kadal_kanakku;
USE kadal_kanakku;


-- 1. BOATS registered at the landing centre
CREATE TABLE boats (
    boat_id      INT PRIMARY KEY,
    boat_name    VARCHAR(30) NOT NULL,
    reg_no       VARCHAR(20) NOT NULL UNIQUE,
    boat_type    VARCHAR(15) NOT NULL,      -- Kattumaram / Vallam / Fibre / Trawler
    owner_name   VARCHAR(30) NOT NULL,
    capacity_kg  INT
);


-- 2. FISHERMEN (each one works on one boat)
CREATE TABLE fishermen (
    fisher_id    INT PRIMARY KEY,
    fname        VARCHAR(30) NOT NULL,
    phone        CHAR(10) UNIQUE,
    village      VARCHAR(25),
    boat_id      INT,
    FOREIGN KEY (boat_id) REFERENCES boats(boat_id)
);


-- 3. TRIPS (one boat going to sea) + sea-safety status
CREATE TABLE trips (
    trip_id      INT PRIMARY KEY,
    boat_id      INT NOT NULL,
    trip_date    DATE NOT NULL,
    crew_count   INT,
    weather      VARCHAR(10),               -- CALM / MODERATE / ROUGH
    status       VARCHAR(10) DEFAULT 'AT SEA', -- AT SEA / RETURNED
    FOREIGN KEY (boat_id) REFERENCES boats(boat_id)
);


-- 4. BUYERS (merchants / exporters / local vendors)
CREATE TABLE buyers (
    buyer_id     INT PRIMARY KEY,
    bname        VARCHAR(30) NOT NULL,
    btype        VARCHAR(15),               -- Exporter / Wholesaler / Vendor
    town         VARCHAR(20),
    phone        CHAR(10)
);


-- 5. CATCHES (fish brought back in one trip, sold in auction)
CREATE TABLE catches (
    catch_id     INT PRIMARY KEY,
    trip_id      INT NOT NULL,
    species      VARCHAR(20) NOT NULL,
    weight_kg    FLOAT NOT NULL,
    buyer_id     INT,                        -- NULL = not yet auctioned
    rate_per_kg  FLOAT,                      -- NULL = not yet auctioned
    FOREIGN KEY (trip_id)  REFERENCES trips(trip_id),
    FOREIGN KEY (buyer_id) REFERENCES buyers(buyer_id)
);


INSERT INTO boats VALUES
(101,'St Antony','TN-15-MO-1021','Fibre','Arul Raj',800),
(102,'Kadal Rani','TN-15-MO-1187','Vallam','Selvin',600),
(103,'Arokia Matha','TN-15-MO-0954','Kattumaram','Jesudasan',250),
(104,'Sea Star','TN-15-MO-1342','Trawler','Benjamin',3000),
(105,'Velankanni','TN-15-MO-1209','Fibre','Michael Raj',900),
(106,'Alaigal','TN-15-MO-0877','Kattumaram','Antony Dhas',200),
(107,'Holy Cross','TN-15-MO-1455','Trawler','Xavier',3500),
(108,'Meenavan','TN-15-MO-1033','Vallam','Robert',650),
```

```
(109,'Muthu Kadal','TN-15-MO-1290','Fibre','Gnana Prakash',850),
(110,'Little Flower','TN-15-MO-0998','Vallam','Stephen',700),
(111,'Our Lady','TN-15-MO-1501','Trawler','Peter Raj',3200),
(112,'Thangam','TN-15-MO-1120','Kattumaram','Sahaya Raj',220);


INSERT INTO fishermen VALUES
(1,'Arul Raj','9443100011','Manavalakurichi',101),
(2,'Selvin','9443100012','Manavalakurichi',102),
(3,'Jesudasan','9443100013','Kadiapattinam',103),
(4,'Benjamin','9443100014','Colachel',104),
(5,'Michael Raj','9443100015','Manavalakurichi',105),
(6,'Antony Dhas','9443100016','Muttom',106),
(7,'Xavier','9443100017','Colachel',107),
(8,'Robert','9443100018','Manavalakurichi',108),
(9,'Gnana Prakash','9443100019','Kadiapattinam',109),
(10,'Stephen','9443100020','Manavalakurichi',110),
(11,'Peter Raj','9443100021','Muttom',111),
(12,'Sahaya Raj','9443100022','Manavalakurichi',112),
(13,'John Bosco','9443100023','Manavalakurichi',101),
(14,'Vincent','9443100024','Colachel',104),
(15,'Ravi','9443100025','Kadiapattinam',107);


INSERT INTO trips VALUES
(1,101,'2026-10-20',4,'CALM','RETURNED'),
(2,102,'2026-10-20',3,'CALM','RETURNED'),
(3,104,'2026-10-20',8,'MODERATE','RETURNED'),
(4,103,'2026-10-21',2,'CALM','RETURNED'),
(5,105,'2026-10-21',4,'MODERATE','RETURNED'),
(6,107,'2026-10-21',9,'MODERATE','RETURNED'),
(7,106,'2026-10-22',2,'CALM','RETURNED'),
(8,108,'2026-10-22',3,'CALM','RETURNED'),
(9,109,'2026-10-22',4,'CALM','RETURNED'),
(10,111,'2026-10-23',8,'MODERATE','RETURNED'),
(11,110,'2026-10-23',3,'CALM','RETURNED'),
(12,101,'2026-10-24',4,'CALM','RETURNED'),
(13,104,'2026-10-24',8,'CALM','RETURNED'),
(14,112,'2026-10-25',2,'MODERATE','AT SEA'),
(15,107,'2026-10-25',9,'MODERATE','AT SEA');


INSERT INTO buyers VALUES
(1,'Kanyakumari Sea Exports','Exporter','Colachel','9842200001'),
(2,'Mary Fish Traders','Wholesaler','Nagercoil','9842200002'),
(3,'Selvi','Vendor','Manavalakurichi','9842200003'),
(4,'Kerala Marine Foods','Exporter','Thiruvananthapuram','9842200004'),
(5,'Jeya Fish Stall','Vendor','Thuckalay','9842200005'),
(6,'Ocean Fresh Agency','Wholesaler','Marthandam','9842200006'),
(7,'Lourdu Mary','Vendor','Kadiapattinam','9842200007'),
(8,'Cape Seafoods','Exporter','Tuticorin','9842200008'),
(9,'Raja Dry Fish','Wholesaler','Colachel','9842200009'),
(10,'Pushpa','Vendor','Muttom','9842200010');


INSERT INTO catches VALUES
(1,1,'Seer Fish',120,1,620),
(2,1,'Mackerel',200,2,140),
(3,2,'Sardine',350,6,60),
(4,3,'Tuna',900,4,210),
(5,3,'Squid',300,1,280),
(6,3,'Prawn',150,8,450),
(7,4,'Anchovy',80,3,90),
(8,5,'Seer Fish',140,8,600),
(9,5,'Pomfret',60,2,520),
(10,6,'Tuna',1100,1,200),
(11,6,'Squid',250,4,290),
(12,7,'Sardine',90,5,70),
(13,8,'Mackerel',260,6,150),
(14,9,'Prawn',110,4,470),
(15,10,'Tuna',800,8,205),
(16,10,'Crab',70,2,380),
(17,11,'Sardine',400,9,55),
(18,12,'Seer Fish',100,1,640),
(19,13,'Squid',320,NULL,NULL),
(20,13,'Anchovy',500,NULL,NULL);
```

7.2 kadal_kanakku.py

```
# ================================================================
#  KADAL KANAKKU  ("Sea Accounts")
#  Fish Catch, Auction & Sea-Safety Manager
```

```
#  for the Manavalakurichi Fish Landing Centre
#
#  CBSE Class XII Computer Science (083) Project, 2026-27
#  Student : Ryan Editraj, XII A, Roll No. 14
#  School  : St Joseph's School (CBSE), Manavalakurichi
#
#  Front end : Python 3 (menu driven console program)
#  Back end  : MySQL  (connected using mysql.connector)
# ====================================================================


import mysql.connector          # Python - MySQL connectivity
import csv                      # to export a report as a CSV file
from datetime import date       # to get today's date


# ---------------- database login details --------------------------
HOST = "localhost"
USER = "root"
PASSWORD = "root"               # <-- change to YOUR MySQL password
DATABASE = "kadal_kanakku"


BOAT_TYPES = ("KATTUMARAM", "VALLAM", "FIBRE", "TRAWLER")
WEATHER = ("CALM", "MODERATE", "ROUGH")


con = None                      # global connection object
cur = None                      # global cursor object



# ================== HELPER FUNCTIONS ==============================
def connect():
    """Connect to MySQL and create the global cursor."""
    global con, cur
    con = mysql.connector.connect(host=HOST, user=USER,
                                  passwd=PASSWORD, database=DATABASE,
                                  charset="utf8mb4",
                                  collation="utf8mb4_general_ci")  # works on MySQL & MariaDB
    cur = con.cursor()



def read_int(msg, low=None, high=None):
    """Keep asking until the user types a valid whole number."""
    while True:
        try:
            n = int(input(msg))
            if (low is not None and n < low) or (high is not None and n > high):
                print("  ! Number must be between", low, "and", high)
            else:
                return n
        except ValueError:
            print("  ! Please enter a whole number.")



def read_float(msg):
    """Keep asking until the user types a positive number."""
    while True:
        try:
            x = float(input(msg))
            if x > 0:
                return x
            print("  ! Value must be greater than 0.")
        except ValueError:
            print("  ! Please enter a number (e.g. 150 or 72.5).")



def read_text(msg):
    """Read a non-empty string."""
    while True:
        s = input(msg).strip()
        if s != "":
            return s
        print("  ! This field cannot be empty.")
```

```
def read_date(msg):
    """Read a date in YYYY-MM-DD form. Empty input = today."""
    while True:
        s = input(msg + " (YYYY-MM-DD, Enter = today): ").strip()
        if s == "":
            return str(date.today())
        try:
            y, m, d = s.split("-")
            date(int(y), int(m), int(d))    # raises error if invalid
            return s
        except ValueError:
            print("  ! Invalid date. Example: 2026-10-24")



def read_choice(msg, options):
    """Read one value from a tuple of allowed options."""
    while True:
        s = input(msg + " " + str(options) + ": ").strip().upper()
        if s in options:
            return s
        print("  ! Choose one of", options)



def exists(table, column, value):
    """Return True if a row with column = value exists in table."""
    cur.execute("SELECT COUNT(*) FROM " + table + " WHERE " + column + " = %s",
                (value,))
    return cur.fetchone()[0] > 0



def show(headings, rows):
    """Print rows as a neat table using plain string formatting."""
    if len(rows) == 0:
        print("  (no records found)")
        return
    # width of each column = longest value in that column
    widths = [len(h) for h in headings]
    for r in rows:
        for i in range(len(r)):
            widths[i] = max(widths[i], len(str(r[i])))
    line = "+" + "+".join("-" * (w + 2) for w in widths) + "+"
    print(line)
    print("| " + " | ".join(headings[i].ljust(widths[i])
                            for i in range(len(headings))) + " |")
    print(line)
    for r in rows:
        print("| " + " | ".join(str(r[i]).ljust(widths[i])
                                for i in range(len(r))) + " |")
    print(line)
    print("  ", len(rows), "row(s)")



def run(query, values=()):
    """Run INSERT / UPDATE / DELETE, commit and return rows affected."""
    try:
        cur.execute(query, values)
        con.commit()
        return cur.rowcount
    except mysql.connector.Error as e:
        con.rollback()
        print("  ! Database error:", e.msg)
        return 0



# ================== 1. BOATS & FISHERMEN ==========================
def add_boat():
    bid = read_int("Boat ID      : ", 1)
    if exists("boats", "boat_id", bid):
        print("  ! Boat ID already exists.")
        return
    name = read_text("Boat name    : ")
    reg = read_text("Reg. number  : ").upper()
```

```
    btype = read_choice("Boat type   ", BOAT_TYPES).title()
    owner = read_text("Owner name   : ")
    cap = read_int("Capacity (kg): ", 50, 10000)
    if run("INSERT INTO boats VALUES (%s,%s,%s,%s,%s,%s)",
           (bid, name, reg, btype, owner, cap)):
        print("  Boat added successfully.")



def view_boats():
    cur.execute("SELECT * FROM boats ORDER BY boat_id")
    show(["ID", "Boat", "Reg No", "Type", "Owner", "Cap(kg)"], cur.fetchall())



def add_fisherman():
    fid = read_int("Fisherman ID : ", 1)
    if exists("fishermen", "fisher_id", fid):
        print("  ! Fisherman ID already exists.")
        return
    name = read_text("Name         : ")
    phone = read_text("Phone (10 digits): ")
    if not (phone.isdigit() and len(phone) == 10):
        print("  ! Phone must be exactly 10 digits.")
        return
    village = read_text("Village      : ")
    bid = read_int("Works on Boat ID: ")
    if not exists("boats", "boat_id", bid):
        print("  ! No such boat.")
        return
    if run("INSERT INTO fishermen VALUES (%s,%s,%s,%s,%s)",
           (fid, name, phone, village, bid)):
        print("  Fisherman added successfully.")



def view_crew():
    """JOIN: list fishermen together with their boat name."""
    cur.execute("""SELECT f.fisher_id, f.fname, f.phone, f.village, b.boat_name
                   FROM fishermen f, boats b
                   WHERE f.boat_id = b.boat_id
                   ORDER BY b.boat_name, f.fname""")
    show(["ID", "Name", "Phone", "Village", "Boat"], cur.fetchall())



def search_fisherman():
    """LIKE search on name or village."""
    word = read_text("Enter part of name / village: ")
    pat = "%" + word + "%"
    cur.execute("""SELECT f.fisher_id, f.fname, f.village, b.boat_name
                   FROM fishermen f JOIN boats b ON f.boat_id = b.boat_id
                   WHERE f.fname LIKE %s OR f.village LIKE %s""", (pat, pat))
    show(["ID", "Name", "Village", "Boat"], cur.fetchall())



def update_owner():
    bid = read_int("Boat ID: ")
    owner = read_text("New owner name: ")
    if run("UPDATE boats SET owner_name = %s WHERE boat_id = %s", (owner, bid)):
        print("  Owner updated.")
    else:
        print("  ! Boat not found.")



def delete_fisherman():
    fid = read_int("Fisherman ID to delete: ")
    if input("  Are you sure? (y/n): ").lower() == "y":
        if run("DELETE FROM fishermen WHERE fisher_id = %s", (fid,)):
            print("  Deleted.")
        else:
            print("  ! Fisherman not found.")



def boat_menu():
```

```
    while True:
        print("\n--- BOATS & FISHERMEN ---")
        print("1.Add boat  2.View boats  3.Add fisherman  4.View crew (join)")
        print("5.Search fisherman  6.Change boat owner  7.Delete fisherman  0.Back")
        ch = read_int("Choice: ", 0, 7)
        if ch == 0:
            break
        [None, add_boat, view_boats, add_fisherman, view_crew,
         search_fisherman, update_owner, delete_fisherman][ch]()



# ================== 2. TRIPS & SEA SAFETY ========================
def start_trip():
    """Log a boat leaving for sea, with a weather safety check."""
    bid = read_int("Boat ID: ")
    cur.execute("SELECT boat_name, boat_type FROM boats WHERE boat_id = %s", (bid,))
    row = cur.fetchone()
    if row is None:
        print("  ! No such boat.")
        return
    # a boat cannot go out again while it is still at sea
    cur.execute("SELECT COUNT(*) FROM trips WHERE boat_id=%s AND status='AT SEA'",
                (bid,))
    if cur.fetchone()[0] > 0:
        print("  !", row[0], "is already at sea.")
        return
    weather = read_choice("Weather", WEATHER)
    # SAFETY RULE: small country boats must not go out in rough sea
    if weather == "ROUGH" and row[1].upper() in ("KATTUMARAM", "VALLAM"):
        print("  *** SAFETY ALERT: Rough sea. A", row[1],
              "is NOT allowed to go out. Trip not logged. ***")
        return
    crew = read_int("Crew count: ", 1, 30)
    tdate = read_date("Trip date")
    cur.execute("SELECT IFNULL(MAX(trip_id),0) + 1 FROM trips")
    tid = cur.fetchone()[0]                 # auto next trip number
    if run("INSERT INTO trips VALUES (%s,%s,%s,%s,%s,'AT SEA')",
           (tid, bid, tdate, crew, weather)):
        print("  Trip", tid, "logged.", row[0], "is now AT SEA.")



def mark_returned():
    tid = read_int("Trip ID that returned: ")
    if run("UPDATE trips SET status='RETURNED' WHERE trip_id=%s AND status='AT SEA'",
           (tid,)):
        print("  Trip", tid, "marked RETURNED. Welcome back!")
    else:
        print("  ! Trip not found or already returned.")



def boats_at_sea():
    """Safety board: who is still at sea and how many people."""
    cur.execute("""SELECT t.trip_id, b.boat_name, b.boat_type, t.trip_date,
                          t.crew_count, t.weather
                   FROM trips t, boats b
                   WHERE t.boat_id = b.boat_id AND t.status = 'AT SEA'
                   ORDER BY t.trip_date""")
    rows = cur.fetchall()
    show(["Trip", "Boat", "Type", "Left On", "Crew", "Weather"], rows)
    total = sum(r[4] for r in rows)
    print("  >> Total people at sea now:", total)



def view_trips():
    cur.execute("""SELECT t.trip_id, t.trip_date, b.boat_name, t.crew_count,
                          t.weather, t.status
                   FROM trips t JOIN boats b ON t.boat_id = b.boat_id
                   ORDER BY t.trip_date DESC, t.trip_id DESC""")
    show(["Trip", "Date", "Boat", "Crew", "Weather", "Status"], cur.fetchall())



def trip_menu():
    while True:
        print("\n--- TRIPS & SEA SAFETY ---")
```

```
        print("1.Start trip  2.Mark returned  3.Boats at sea  4.All trips  0.Back")
        ch = read_int("Choice: ", 0, 4)
        if ch == 0:
            break
        [None, start_trip, mark_returned, boats_at_sea, view_trips][ch]()



# ================== 3. CATCH & AUCTION ============================
def record_catch():
    tid = read_int("Trip ID: ")
    cur.execute("SELECT status FROM trips WHERE trip_id = %s", (tid,))
    row = cur.fetchone()
    if row is None:
        print("  ! No such trip.")
        return
    if row[0] != "RETURNED":
        print("  ! Boat has not returned yet. Mark the trip returned first.")
        return
    species = read_text("Species (e.g. Tuna): ").title()
    kg = read_float("Weight in kg: ")
    cur.execute("SELECT IFNULL(MAX(catch_id),0) + 1 FROM catches")
    cid = cur.fetchone()[0]
    if run("INSERT INTO catches (catch_id, trip_id, species, weight_kg) "
           "VALUES (%s,%s,%s,%s)", (cid, tid, species, kg)):
        print("  Catch", cid, "recorded. Waiting for auction.")



def unsold_catches():
    cur.execute("""SELECT c.catch_id, b.boat_name, t.trip_date, c.species, c.weight_kg
                   FROM catches c, trips t, boats b
                   WHERE c.trip_id = t.trip_id AND t.boat_id = b.boat_id
                     AND c.buyer_id IS NULL
                   ORDER BY c.catch_id""")
    show(["Catch", "Boat", "Date", "Species", "Kg"], cur.fetchall())



def auction_catch():
    """Sell an unsold catch to the highest bidder."""
    unsold_catches()
    cid = read_int("Catch ID to auction: ")
    cur.execute("SELECT species, weight_kg FROM catches "
                "WHERE catch_id=%s AND buyer_id IS NULL", (cid,))
    row = cur.fetchone()
    if row is None:
        print("  ! Not found or already sold.")
        return
    bid = read_int("Winning Buyer ID: ")
    if not exists("buyers", "buyer_id", bid):
        print("  ! No such buyer.")
        return
    rate = read_float("Final bid rate (Rs per kg): ")
    if run("UPDATE catches SET buyer_id=%s, rate_per_kg=%s WHERE catch_id=%s",
           (bid, rate, cid)):
        print("  SOLD!", row[1], "kg of", row[0], "@ Rs", rate,
              "= Rs", round(row[1] * rate, 2))



def view_sales():
    cur.execute("""SELECT c.catch_id, t.trip_date, bo.boat_name, c.species,
                          c.weight_kg, c.rate_per_kg,
                          c.weight_kg * c.rate_per_kg, bu.bname
                   FROM catches c
                   JOIN trips t   ON c.trip_id = t.trip_id
                   JOIN boats bo  ON t.boat_id = bo.boat_id
                   JOIN buyers bu ON c.buyer_id = bu.buyer_id
                   ORDER BY t.trip_date, c.catch_id""")
    show(["Catch", "Date", "Boat", "Species", "Kg", "Rate", "Amount", "Buyer"],
         cur.fetchall())



def delete_catch():
    cid = read_int("Catch ID to delete (wrong entry): ")
    if run("DELETE FROM catches WHERE catch_id = %s", (cid,)):
        print("  Deleted.")
```

```
    else:
        print("  ! Catch not found.")



def catch_menu():
    while True:
        print("\n--- CATCH & AUCTION ---")
        print("1.Record catch  2.Unsold catches  3.Auction a catch")
        print("4.View all sales  5.Delete catch  0.Back")
        ch = read_int("Choice: ", 0, 5)
        if ch == 0:
            break
        [None, record_catch, unsold_catches, auction_catch,
         view_sales, delete_catch][ch]()



# ================== 4. BUYERS ====================================
def add_buyer():
    bid = read_int("Buyer ID: ", 1)
    if exists("buyers", "buyer_id", bid):
        print("  ! Buyer ID already exists.")
        return
    name = read_text("Name: ")
    btype = read_choice("Type", ("EXPORTER", "WHOLESALER", "VENDOR")).title()
    town = read_text("Town: ")
    phone = read_text("Phone: ")
    if run("INSERT INTO buyers VALUES (%s,%s,%s,%s,%s)",
           (bid, name, btype, town, phone)):
        print("  Buyer added.")



def view_buyers():
    cur.execute("SELECT * FROM buyers ORDER BY btype, bname")
    show(["ID", "Name", "Type", "Town", "Phone"], cur.fetchall())



def buyer_menu():
    while True:
        print("\n--- BUYERS ---")
        print("1.Add buyer  2.View buyers  0.Back")
        ch = read_int("Choice: ", 0, 2)
        if ch == 0:
            break
        [None, add_buyer, view_buyers][ch]()



# ================== 5. REPORTS ====================================
def daily_report():
    d = read_date("Report date")
    cur.execute("""SELECT c.species, SUM(c.weight_kg), SUM(c.weight_kg*c.rate_per_kg)
                   FROM catches c JOIN trips t ON c.trip_id = t.trip_id
                   WHERE t.trip_date = %s AND c.buyer_id IS NOT NULL
                   GROUP BY c.species ORDER BY 3 DESC""", (d,))
    rows = cur.fetchall()
    show(["Species", "Total Kg", "Revenue (Rs)"], rows)
    print("  >> Total auction revenue on", d, ": Rs", sum(r[2] for r in rows))



def species_report():
    """GROUP BY species with aggregate functions. Returns rows for CSV."""
    cur.execute("""SELECT species, COUNT(*), SUM(weight_kg),
                          ROUND(AVG(rate_per_kg),2), MAX(rate_per_kg),
                          SUM(weight_kg*rate_per_kg)
                   FROM catches WHERE buyer_id IS NOT NULL
                   GROUP BY species ORDER BY SUM(weight_kg*rate_per_kg) DESC""")
    rows = cur.fetchall()
    show(["Species", "Lots", "Total Kg", "Avg Rate", "Max Rate", "Revenue"], rows)
    return rows



def top_boats():
```

```
    cur.execute("""SELECT b.boat_name, b.boat_type, COUNT(DISTINCT t.trip_id),
                          SUM(c.weight_kg), SUM(c.weight_kg*c.rate_per_kg)
                   FROM boats b, trips t, catches c
                   WHERE b.boat_id = t.boat_id AND t.trip_id = c.trip_id
                     AND c.buyer_id IS NOT NULL
                   GROUP BY b.boat_id, b.boat_name, b.boat_type
                   ORDER BY 5 DESC LIMIT 5""")
    rows = cur.fetchall()
    show(["Boat", "Type", "Trips", "Kg", "Earnings (Rs)"], rows)
    if rows:
        print("  >> TOP BOAT:", rows[0][0])



def buyer_report():
    """GROUP BY + HAVING: buyers who spent more than a given amount."""
    amt = read_float("Show buyers who spent more than Rs: ")
    cur.execute("""SELECT bu.bname, bu.btype, COUNT(*),
                          SUM(c.weight_kg*c.rate_per_kg)
                   FROM buyers bu, catches c
                   WHERE bu.buyer_id = c.buyer_id
                   GROUP BY bu.buyer_id, bu.bname, bu.btype
                   HAVING SUM(c.weight_kg*c.rate_per_kg) > %s
                   ORDER BY 4 DESC""", (amt,))
    show(["Buyer", "Type", "Lots", "Spent (Rs)"], cur.fetchall())



def period_report():
    d1 = read_date("From date")
    d2 = read_date("To date  ")
    cur.execute("""SELECT t.trip_date, COUNT(DISTINCT t.trip_id),
                          SUM(c.weight_kg), SUM(c.weight_kg*c.rate_per_kg)
                   FROM trips t JOIN catches c ON t.trip_id = c.trip_id
                   WHERE t.trip_date BETWEEN %s AND %s AND c.buyer_id IS NOT NULL
                   GROUP BY t.trip_date ORDER BY t.trip_date""", (d1, d2))
    show(["Date", "Trips", "Kg", "Revenue (Rs)"], cur.fetchall())



def export_csv():
    """Write the species report into a CSV file using the csv module."""
    rows = species_report()
    fname = "species_report.csv"
    try:
        with open(fname, "w", newline="") as f:
            w = csv.writer(f)
            w.writerow(["Species", "Lots", "Total_Kg", "Avg_Rate",
                        "Max_Rate", "Revenue"])
            w.writerows(rows)
        print("  Report saved to", fname)
        # read it back to prove it worked
        with open(fname, "r") as f:
            print("  First 3 lines of the file:")
            for i, line in enumerate(csv.reader(f)):
                if i == 3:
                    break
                print("   ", line)
    except IOError:
        print("  ! Could not write the file.")



def report_menu():
    while True:
        print("\n--- REPORTS ---")
        print("1.Daily revenue  2.Species-wise totals  3.Top 5 boats")
        print("4.Buyer spending (HAVING)  5.Revenue between dates")
        print("6.Export species report to CSV  0.Back")
        ch = read_int("Choice: ", 0, 6)
        if ch == 0:
            break
        [None, daily_report, species_report, top_boats,
         buyer_report, period_report, export_csv][ch]()



# ================== MAIN PROGRAM ===================================
def main():
```

```
    print("=" * 60)
    print("   KADAL KANAKKU - Fish Catch, Auction & Sea-Safety Manager")
    print("        Manavalakurichi Fish Landing Centre")
    print("=" * 60)
    try:
        connect()
    except mysql.connector.Error as e:
        print("Cannot connect to MySQL:", e.msg)
        print("Check that MySQL is running, schema.sql is loaded and")
        print("PASSWORD at the top of this file is correct.")
        return
    try:
        while True:
            print("\n========= MAIN MENU =========")
            print("1. Boats & Fishermen")
            print("2. Trips & Sea Safety")
            print("3. Catch & Auction")
            print("4. Buyers")
            print("5. Reports")
            print("0. Exit")
            ch = read_int("Enter choice: ", 0, 5)
            if ch == 0:
                break
            [None, boat_menu, trip_menu, catch_menu, buyer_menu, report_menu][ch]()
    finally:
        con.close()                # always close the connection
        print("\nConnection closed. Nanri! (Thank you)")



if __name__ == "__main__":
    main()
```

8. Sample Outputs

The following outputs were captured from actual runs of the program against the sample database (user input is shown after each prompt).

8.1 Boats & Fishermen menu

```
============================================================
   KADAL KANAKKU - Fish Catch, Auction & Sea-Safety Manager
        Manavalakurichi Fish Landing Centre
============================================================


========= MAIN MENU =========
1. Boats & Fishermen
2. Trips & Sea Safety
3. Catch & Auction
4. Buyers
5. Reports
0. Exit
Enter choice: 1


--- BOATS & FISHERMEN ---
1.Add boat  2.View boats  3.Add fisherman  4.View crew (join)
5.Search fisherman  6.Change boat owner  7.Delete fisherman  0.Back
Choice: 2
+-----+---------------+---------------+------------+---------------+---------+
| ID  | Boat          | Reg No        | Type       | Owner         | Cap(kg) |
+-----+---------------+---------------+------------+---------------+---------+
| 101 | St Antony     | TN-15-MO-1021 | Fibre      | Arul Raj      | 800     |
| 102 | Kadal Rani    | TN-15-MO-1187 | Vallam     | Selvin        | 600     |
| 103 | Arokia Matha  | TN-15-MO-0954 | Kattumaram | Jesudasan     | 250     |
| 104 | Sea Star      | TN-15-MO-1342 | Trawler    | Benjamin      | 3000    |
| 105 | Velankanni    | TN-15-MO-1209 | Fibre      | Michael Raj   | 900     |
| 106 | Alaigal       | TN-15-MO-0877 | Kattumaram | Antony Dhas   | 200     |
| 107 | Holy Cross    | TN-15-MO-1455 | Trawler    | Xavier        | 3500    |
| 108 | Meenavan      | TN-15-MO-1033 | Vallam     | Robert        | 650     |
| 109 | Muthu Kadal   | TN-15-MO-1290 | Fibre      | Gnana Prakash | 850     |
| 110 | Little Flower | TN-15-MO-0998 | Vallam     | Stephen       | 700     |
| 111 | Our Lady      | TN-15-MO-1501 | Trawler    | Peter Raj     | 3200    |
| 112 | Thangam       | TN-15-MO-1120 | Kattumaram | Sahaya Raj    | 220     |
+-----+---------------+---------------+------------+---------------+---------+
   12 row(s)


--- BOATS & FISHERMEN ---
1.Add boat  2.View boats  3.Add fisherman  4.View crew (join)
5.Search fisherman  6.Change boat owner  7.Delete fisherman  0.Back
Choice: 4
+----+---------------+------------+-----------------+---------------+
| ID | Name          | Phone      | Village         | Boat          |
+----+---------------+------------+-----------------+---------------+
| 6  | Antony Dhas   | 9443100016 | Muttom          | Alaigal       |
| 3  | Jesudasan     | 9443100013 | Kadiapattinam   | Arokia Matha  |
| 15 | Ravi          | 9443100025 | Kadiapattinam   | Holy Cross    |
| 7  | Xavier        | 9443100017 | Colachel        | Holy Cross    |
| 2  | Selvin        | 9443100012 | Manavalakurichi | Kadal Rani    |
| 10 | Stephen       | 9443100020 | Manavalakurichi | Little Flower |
| 8  | Robert        | 9443100018 | Manavalakurichi | Meenavan      |
| 9  | Gnana Prakash | 9443100019 | Kadiapattinam   | Muthu Kadal   |
| 11 | Peter Raj     | 9443100021 | Muttom          | Our Lady      |
| 4  | Benjamin      | 9443100014 | Colachel        | Sea Star      |
| 14 | Vincent       | 9443100024 | Colachel        | Sea Star      |
| 1  | Arul Raj      | 9443100011 | Manavalakurichi | St Antony     |
| 13 | John Bosco    | 9443100023 | Manavalakurichi | St Antony     |
| 12 | Sahaya Raj    | 9443100022 | Manavalakurichi | Thangam       |
| 5  | Michael Raj   | 9443100015 | Manavalakurichi | Velankanni    |
+----+---------------+------------+-----------------+---------------+
   15 row(s)


--- BOATS & FISHERMEN ---
1.Add boat  2.View boats  3.Add fisherman  4.View crew (join)
5.Search fisherman  6.Change boat owner  7.Delete fisherman  0.Back
Choice: 5
Enter part of name / village: bosco
+----+------------+-----------------+-----------+
| ID | Name       | Village         | Boat      |
+----+------------+-----------------+-----------+
| 13 | John Bosco | Manavalakurichi | St Antony |
+----+------------+-----------------+-----------+
   1 row(s)
```

```
--- BOATS & FISHERMEN ---
1.Add boat  2.View boats  3.Add fisherman  4.View crew (join)
5.Search fisherman  6.Change boat owner  7.Delete fisherman  0.Back
Choice: 1
Boat ID      : 113
Boat name    : Sea Breeze
Reg. number  : tn-15-mo-1600
Boat type    ('KATTUMARAM', 'VALLAM', 'FIBRE', 'TRAWLER'): abc
  ! Choose one of ('KATTUMARAM', 'VALLAM', 'FIBRE', 'TRAWLER')
Boat type    ('KATTUMARAM', 'VALLAM', 'FIBRE', 'TRAWLER'): fibre
Owner name   : Antony Raj
Capacity (kg): 900
  Boat added successfully.


--- BOATS & FISHERMEN ---
1.Add boat  2.View boats  3.Add fisherman  4.View crew (join)
5.Search fisherman  6.Change boat owner  7.Delete fisherman  0.Back
Choice: 1
Boat ID      : 101
  ! Boat ID already exists.


--- BOATS & FISHERMEN ---
1.Add boat  2.View boats  3.Add fisherman  4.View crew (join)
5.Search fisherman  6.Change boat owner  7.Delete fisherman  0.Back
Choice: 6
Boat ID: 103
New owner name: Jesudasan Jr
  Owner updated.


--- BOATS & FISHERMEN ---
1.Add boat  2.View boats  3.Add fisherman  4.View crew (join)
5.Search fisherman  6.Change boat owner  7.Delete fisherman  0.Back
Choice: 3
Fisherman ID : 16
Name         : Leo
Phone (10 digits): 98765
  ! Phone must be exactly 10 digits.


--- BOATS & FISHERMEN ---
1.Add boat  2.View boats  3.Add fisherman  4.View crew (join)
5.Search fisherman  6.Change boat owner  7.Delete fisherman  0.Back
Choice: 3
Fisherman ID : 17
Name         : Leo Raj
Phone (10 digits): 9876543210
Village      : Manavalakurichi
Works on Boat ID: 113
  Fisherman added successfully.


--- BOATS & FISHERMEN ---
1.Add boat  2.View boats  3.Add fisherman  4.View crew (join)
5.Search fisherman  6.Change boat owner  7.Delete fisherman  0.Back
Choice: 4
+----+---------------+------------+-----------------+---------------+
| ID | Name          | Phone      | Village         | Boat          |
+----+---------------+------------+-----------------+---------------+
| 6  | Antony Dhas   | 9443100016 | Muttom          | Alaigal       |
| 3  | Jesudasan     | 9443100013 | Kadiapattinam   | Arokia Matha  |
| 15 | Ravi          | 9443100025 | Kadiapattinam   | Holy Cross    |
| 7  | Xavier        | 9443100017 | Colachel        | Holy Cross    |
| 2  | Selvin        | 9443100012 | Manavalakurichi | Kadal Rani    |
| 10 | Stephen       | 9443100020 | Manavalakurichi | Little Flower |
| 8  | Robert        | 9443100018 | Manavalakurichi | Meenavan      |
| 9  | Gnana Prakash | 9443100019 | Kadiapattinam   | Muthu Kadal   |
| 11 | Peter Raj     | 9443100021 | Muttom          | Our Lady      |
| 17 | Leo Raj       | 9876543210 | Manavalakurichi | Sea Breeze    |
| 4  | Benjamin      | 9443100014 | Colachel        | Sea Star      |
| 14 | Vincent       | 9443100024 | Colachel        | Sea Star      |
| 1  | Arul Raj      | 9443100011 | Manavalakurichi | St Antony     |
| 13 | John Bosco    | 9443100023 | Manavalakurichi | St Antony     |
| 12 | Sahaya Raj    | 9443100022 | Manavalakurichi | Thangam       |
| 5  | Michael Raj   | 9443100015 | Manavalakurichi | Velankanni    |
+----+---------------+------------+-----------------+---------------+
   16 row(s)


--- BOATS & FISHERMEN ---
1.Add boat  2.View boats  3.Add fisherman  4.View crew (join)
5.Search fisherman  6.Change boat owner  7.Delete fisherman  0.Back
Choice: 7
Fisherman ID to delete: 17
  Are you sure? (y/n): y
```

```
  Deleted.


--- BOATS & FISHERMEN ---
1.Add boat  2.View boats  3.Add fisherman  4.View crew (join)
5.Search fisherman  6.Change boat owner  7.Delete fisherman  0.Back
Choice: 0


========= MAIN MENU =========
1. Boats & Fishermen
2. Trips & Sea Safety
3. Catch & Auction
4. Buyers
5. Reports
0. Exit
Enter choice: 0


Connection closed. Nanri! (Thank you)
```

8.2 Trips & Sea Safety menu (safety alert)

```
============================================================
   KADAL KANAKKU - Fish Catch, Auction & Sea-Safety Manager
         Manavalakurichi Fish Landing Centre
============================================================


========= MAIN MENU =========
1. Boats & Fishermen
2. Trips & Sea Safety
3. Catch & Auction
4. Buyers
5. Reports
0. Exit
Enter choice: 2


--- TRIPS & SEA SAFETY ---
1.Start trip  2.Mark returned  3.Boats at sea  4.All trips  0.Back
Choice: 3
+------+------------+------------+------------+------+----------+
| Trip | Boat       | Type       | Left On    | Crew | Weather  |
+------+------------+------------+------------+------+----------+
| 14   | Thangam    | Kattumaram | 2026-10-25 | 2    | MODERATE |
| 15   | Holy Cross | Trawler    | 2026-10-25 | 9    | MODERATE |
+------+------------+------------+------------+------+----------+
   2 row(s)
  >> Total people at sea now: 11


--- TRIPS & SEA SAFETY ---
1.Start trip  2.Mark returned  3.Boats at sea  4.All trips  0.Back
Choice: 1
Boat ID: 106
Weather ('CALM', 'MODERATE', 'ROUGH'): ROUGH
  *** SAFETY ALERT: Rough sea. A Kattumaram is NOT allowed to go out. Trip not logged. ***


--- TRIPS & SEA SAFETY ---
1.Start trip  2.Mark returned  3.Boats at sea  4.All trips  0.Back
Choice: 1
Boat ID: 113
Weather ('CALM', 'MODERATE', 'ROUGH'): storm
  ! Choose one of ('CALM', 'MODERATE', 'ROUGH')
Weather ('CALM', 'MODERATE', 'ROUGH'): ROUGH
Crew count: 6
Trip date (YYYY-MM-DD, Enter = today): 2026-10-26
  Trip 16 logged. Sea Breeze is now AT SEA.


--- TRIPS & SEA SAFETY ---
1.Start trip  2.Mark returned  3.Boats at sea  4.All trips  0.Back
Choice: 1
Boat ID: 112
  ! Thangam is already at sea.


--- TRIPS & SEA SAFETY ---
1.Start trip  2.Mark returned  3.Boats at sea  4.All trips  0.Back
Choice: 2
Trip ID that returned: 14
  Trip 14 marked RETURNED. Welcome back!


--- TRIPS & SEA SAFETY ---
1.Start trip  2.Mark returned  3.Boats at sea  4.All trips  0.Back
```

```
Choice: 3
+------+------------+---------+------------+------+----------+
| Trip | Boat       | Type    | Left On    | Crew | Weather  |
+------+------------+---------+------------+------+----------+
| 15   | Holy Cross | Trawler | 2026-10-25 | 9    | MODERATE |
| 16   | Sea Breeze | Fibre   | 2026-10-26 | 6    | ROUGH    |
+------+------------+---------+------------+------+----------+
   2 row(s)
  >> Total people at sea now: 15


--- TRIPS & SEA SAFETY ---
1.Start trip  2.Mark returned  3.Boats at sea  4.All trips  0.Back
Choice: 16
  ! Number must be between 0 and 4
Choice: 2
Trip ID that returned: 99
  ! Trip not found or already returned.


--- TRIPS & SEA SAFETY ---
1.Start trip  2.Mark returned  3.Boats at sea  4.All trips  0.Back
Choice: 4
+------+------------+---------------+------+----------+----------+
| Trip | Date       | Boat          | Crew | Weather  | Status   |
+------+------------+---------------+------+----------+----------+
| 16   | 2026-10-26 | Sea Breeze    | 6    | ROUGH    | AT SEA   |
| 15   | 2026-10-25 | Holy Cross    | 9    | MODERATE | AT SEA   |
| 14   | 2026-10-25 | Thangam       | 2    | MODERATE | RETURNED |
| 13   | 2026-10-24 | Sea Star      | 8    | CALM     | RETURNED |
| 12   | 2026-10-24 | St Antony     | 4    | CALM     | RETURNED |
| 11   | 2026-10-23 | Little Flower | 3    | CALM     | RETURNED |
| 10   | 2026-10-23 | Our Lady      | 8    | MODERATE | RETURNED |
| 9    | 2026-10-22 | Muthu Kadal   | 4    | CALM     | RETURNED |
| 8    | 2026-10-22 | Meenavan      | 3    | CALM     | RETURNED |
| 7    | 2026-10-22 | Alaigal       | 2    | CALM     | RETURNED |
| 6    | 2026-10-21 | Holy Cross    | 9    | MODERATE | RETURNED |
| 5    | 2026-10-21 | Velankanni    | 4    | MODERATE | RETURNED |
| 4    | 2026-10-21 | Arokia Matha  | 2    | CALM     | RETURNED |
| 3    | 2026-10-20 | Sea Star      | 8    | MODERATE | RETURNED |
| 2    | 2026-10-20 | Kadal Rani    | 3    | CALM     | RETURNED |
| 1    | 2026-10-20 | St Antony     | 4    | CALM     | RETURNED |
+------+------------+---------------+------+----------+----------+
   16 row(s)


--- TRIPS & SEA SAFETY ---
1.Start trip  2.Mark returned  3.Boats at sea  4.All trips  0.Back
Choice: 0


========= MAIN MENU =========
1. Boats & Fishermen
2. Trips & Sea Safety
3. Catch & Auction
4. Buyers
5. Reports
0. Exit
Enter choice: 0


Connection closed. Nanri! (Thank you)
```

8.3 Catch & Auction menu

```
============================================================
   KADAL KANAKKU - Fish Catch, Auction & Sea-Safety Manager
        Manavalakurichi Fish Landing Centre
============================================================


========= MAIN MENU =========
1. Boats & Fishermen
2. Trips & Sea Safety
3. Catch & Auction
4. Buyers
5. Reports
0. Exit
Enter choice: 3


--- CATCH & AUCTION ---
1.Record catch  2.Unsold catches  3.Auction a catch
4.View all sales  5.Delete catch  0.Back
Choice: 1
Trip ID: 16
  ! Boat has not returned yet. Mark the trip returned first.
```

```
--- CATCH & AUCTION ---
1.Record catch  2.Unsold catches  3.Auction a catch
4.View all sales  5.Delete catch  0.Back
Choice: 1
Trip ID: 14
Species (e.g. Tuna): Prawn
Weight in kg: 120
  Catch 21 recorded. Waiting for auction.


--- CATCH & AUCTION ---
1.Record catch  2.Unsold catches  3.Auction a catch
4.View all sales  5.Delete catch  0.Back
Choice: lobster
  ! Please enter a whole number.
Choice: -5
  ! Number must be between 0 and 5
Choice: 2
+-------+----------+------------+---------+-------+
| Catch | Boat     | Date       | Species | Kg    |
+-------+----------+------------+---------+-------+
| 19    | Sea Star | 2026-10-24 | Squid   | 320.0 |
| 20    | Sea Star | 2026-10-24 | Anchovy | 500.0 |
| 21    | Thangam  | 2026-10-25 | Prawn   | 120.0 |
+-------+----------+------------+---------+-------+
   3 row(s)


--- CATCH & AUCTION ---
1.Record catch  2.Unsold catches  3.Auction a catch
4.View all sales  5.Delete catch  0.Back
Choice: 3
+-------+----------+------------+---------+-------+
| Catch | Boat     | Date       | Species | Kg    |
+-------+----------+------------+---------+-------+
| 19    | Sea Star | 2026-10-24 | Squid   | 320.0 |
| 20    | Sea Star | 2026-10-24 | Anchovy | 500.0 |
| 21    | Thangam  | 2026-10-25 | Prawn   | 120.0 |
+-------+----------+------------+---------+-------+
   3 row(s)
Catch ID to auction: 19
Winning Buyer ID: 3
Final bid rate (Rs per kg): 280
  SOLD! 320.0 kg of Squid @ Rs 280.0 = Rs 89600.0


--- CATCH & AUCTION ---
1.Record catch  2.Unsold catches  3.Auction a catch
4.View all sales  5.Delete catch  0.Back
Choice: 3
+-------+----------+------------+---------+-------+
| Catch | Boat     | Date       | Species | Kg    |
+-------+----------+------------+---------+-------+
| 20    | Sea Star | 2026-10-24 | Anchovy | 500.0 |
| 21    | Thangam  | 2026-10-25 | Prawn   | 120.0 |
+-------+----------+------------+---------+-------+
   2 row(s)
Catch ID to auction: 20
Winning Buyer ID: 99
  ! No such buyer.


--- CATCH & AUCTION ---
1.Record catch  2.Unsold catches  3.Auction a catch
4.View all sales  5.Delete catch  0.Back
Choice: 3
+-------+----------+------------+---------+-------+
| Catch | Boat     | Date       | Species | Kg    |
+-------+----------+------------+---------+-------+
| 20    | Sea Star | 2026-10-24 | Anchovy | 500.0 |
| 21    | Thangam  | 2026-10-25 | Prawn   | 120.0 |
+-------+----------+------------+---------+-------+
   2 row(s)
Catch ID to auction: 20
Winning Buyer ID: 9
Final bid rate (Rs per kg): 95
  SOLD! 500.0 kg of Anchovy @ Rs 95.0 = Rs 47500.0


--- CATCH & AUCTION ---
1.Record catch  2.Unsold catches  3.Auction a catch
4.View all sales  5.Delete catch  0.Back
Choice: 3
+-------+---------+------------+---------+-------+
| Catch | Boat    | Date       | Species | Kg    |
+-------+---------+------------+---------+-------+
```

```
| 21    | Thangam | 2026-10-25 | Prawn   | 120.0 |
+-------+---------+------------+---------+-------+
   1 row(s)
Catch ID to auction: 21
Winning Buyer ID: 3
Final bid rate (Rs per kg): abc
  ! Please enter a number (e.g. 150 or 72.5).
Final bid rate (Rs per kg): -10
  ! Value must be greater than 0.
Final bid rate (Rs per kg): 460
  SOLD! 120.0 kg of Prawn @ Rs 460.0 = Rs 55200.0


--- CATCH & AUCTION ---
1.Record catch  2.Unsold catches  3.Auction a catch
4.View all sales  5.Delete catch  0.Back
Choice: 1
Trip ID: 14
Species (e.g. Tuna): Crab
Weight in kg: 12
  Catch 22 recorded. Waiting for auction.


--- CATCH & AUCTION ---
1.Record catch  2.Unsold catches  3.Auction a catch
4.View all sales  5.Delete catch  0.Back
Choice: 5
Catch ID to delete (wrong entry): 22
  Deleted.


--- CATCH & AUCTION ---
1.Record catch  2.Unsold catches  3.Auction a catch
4.View all sales  5.Delete catch  0.Back
Choice: 5
Catch ID to delete (wrong entry): 99
  ! Catch not found.


--- CATCH & AUCTION ---
1.Record catch  2.Unsold catches  3.Auction a catch
4.View all sales  5.Delete catch  0.Back
Choice: 4
+-------+------------+---------------+-----------+--------+-------+----------+-------------------------+
| Catch | Date       | Boat          | Species   | Kg     | Rate  | Amount   | Buyer                   |
+-------+------------+---------------+-----------+--------+-------+----------+-------------------------+
| 1     | 2026-10-20 | St Antony     | Seer Fish | 120.0  | 620.0 | 74400.0  | Kanyakumari Sea Exports |
| 2     | 2026-10-20 | St Antony     | Mackerel  | 200.0  | 140.0 | 28000.0  | Mary Fish Traders       |
| 3     | 2026-10-20 | Kadal Rani    | Sardine   | 350.0  | 60.0  | 21000.0  | Ocean Fresh Agency      |
| 4     | 2026-10-20 | Sea Star      | Tuna      | 900.0  | 210.0 | 189000.0 | Kerala Marine Foods     |
| 5     | 2026-10-20 | Sea Star      | Squid     | 300.0  | 280.0 | 84000.0  | Kanyakumari Sea Exports |
| 6     | 2026-10-20 | Sea Star      | Prawn     | 150.0  | 450.0 | 67500.0  | Cape Seafoods           |
| 7     | 2026-10-21 | Arokia Matha  | Anchovy   | 80.0   | 90.0  | 7200.0   | Selvi                   |
| 8     | 2026-10-21 | Velankanni    | Seer Fish | 140.0  | 600.0 | 84000.0  | Cape Seafoods           |
| 9     | 2026-10-21 | Velankanni    | Pomfret   | 60.0   | 520.0 | 31200.0  | Mary Fish Traders       |
| 10    | 2026-10-21 | Holy Cross    | Tuna      | 1100.0 | 200.0 | 220000.0 | Kanyakumari Sea Exports |
| 11    | 2026-10-21 | Holy Cross    | Squid     | 250.0  | 290.0 | 72500.0  | Kerala Marine Foods     |
| 12    | 2026-10-22 | Alaigal       | Sardine   | 90.0   | 70.0  | 6300.0   | Jeya Fish Stall         |
| 13    | 2026-10-22 | Meenavan      | Mackerel  | 260.0  | 150.0 | 39000.0  | Ocean Fresh Agency      |
| 14    | 2026-10-22 | Muthu Kadal   | Prawn     | 110.0  | 470.0 | 51700.0  | Kerala Marine Foods     |
| 15    | 2026-10-23 | Our Lady      | Tuna      | 800.0  | 205.0 | 164000.0 | Cape Seafoods           |
| 16    | 2026-10-23 | Our Lady      | Crab      | 70.0   | 380.0 | 26600.0  | Mary Fish Traders       |
| 17    | 2026-10-23 | Little Flower | Sardine   | 400.0  | 55.0  | 22000.0  | Raja Dry Fish           |
| 18    | 2026-10-24 | St Antony     | Seer Fish | 100.0  | 640.0 | 64000.0  | Kanyakumari Sea Exports |
| 19    | 2026-10-24 | Sea Star      | Squid     | 320.0  | 280.0 | 89600.0  | Selvi                   |
| 20    | 2026-10-24 | Sea Star      | Anchovy   | 500.0  | 95.0  | 47500.0  | Raja Dry Fish           |
| 21    | 2026-10-25 | Thangam       | Prawn     | 120.0  | 460.0 | 55200.0  | Selvi                   |
+-------+------------+---------------+-----------+--------+-------+----------+-------------------------+
   21 row(s)


--- CATCH & AUCTION ---
1.Record catch  2.Unsold catches  3.Auction a catch
4.View all sales  5.Delete catch  0.Back
Choice: 0


========= MAIN MENU =========
1. Boats & Fishermen
2. Trips & Sea Safety
3. Catch & Auction
4. Buyers
5. Reports
0. Exit
Enter choice: 0


Connection closed. Nanri! (Thank you)
```

8.4 Buyers menu

```
============================================================
   KADAL KANAKKU - Fish Catch, Auction & Sea-Safety Manager
        Manavalakurichi Fish Landing Centre
============================================================


========= MAIN MENU =========
1. Boats & Fishermen
2. Trips & Sea Safety
3. Catch & Auction
4. Buyers
5. Reports
0. Exit
Enter choice: 4


--- BUYERS ---
1.Add buyer  2.View buyers  0.Back
Choice: 1
Buyer ID: 11
Name: Sahaya Fisheries
Type ('EXPORTER', 'WHOLESALER', 'VENDOR'): retailer
  ! Choose one of ('EXPORTER', 'WHOLESALER', 'VENDOR')
Type ('EXPORTER', 'WHOLESALER', 'VENDOR'): wholesaler
Town: Nagercoil
Phone: 9842200011
  Buyer added.


--- BUYERS ---
1.Add buyer  2.View buyers  0.Back
Choice: 2
+----+-------------------------+------------+--------------------+------------+
| ID | Name                    | Type       | Town               | Phone      |
+----+-------------------------+------------+--------------------+------------+
| 8  | Cape Seafoods           | Exporter   | Tuticorin          | 9842200008 |
| 1  | Kanyakumari Sea Exports | Exporter   | Colachel           | 9842200001 |
| 4  | Kerala Marine Foods     | Exporter   | Thiruvananthapuram | 9842200004 |
| 5  | Jeya Fish Stall         | Vendor     | Thuckalay          | 9842200005 |
| 7  | Lourdu Mary             | Vendor     | Kadiapattinam      | 9842200007 |
| 10 | Pushpa                  | Vendor     | Muttom             | 9842200010 |
| 3  | Selvi                   | Vendor     | Manavalakurichi    | 9842200003 |
| 2  | Mary Fish Traders       | Wholesaler | Nagercoil          | 9842200002 |
| 6  | Ocean Fresh Agency      | Wholesaler | Marthandam         | 9842200006 |
| 9  | Raja Dry Fish           | Wholesaler | Colachel           | 9842200009 |
| 11 | Sahaya Fisheries        | Wholesaler | Nagercoil          | 9842200011 |
+----+-------------------------+------------+--------------------+------------+
   11 row(s)


--- BUYERS ---
1.Add buyer  2.View buyers  0.Back
Choice: 0


========= MAIN MENU =========
1. Boats & Fishermen
2. Trips & Sea Safety
3. Catch & Auction
4. Buyers
5. Reports
0. Exit
Enter choice: 0


Connection closed. Nanri! (Thank you)
```

8.5 Reports menu and CSV export

```
============================================================
   KADAL KANAKKU - Fish Catch, Auction & Sea-Safety Manager
        Manavalakurichi Fish Landing Centre
============================================================


========= MAIN MENU =========
1. Boats & Fishermen
2. Trips & Sea Safety
3. Catch & Auction
4. Buyers
5. Reports
0. Exit
Enter choice: 5
```

```
--- REPORTS ---
1.Daily revenue  2.Species-wise totals  3.Top 5 boats
4.Buyer spending (HAVING)  5.Revenue between dates
6.Export species report to CSV  0.Back
Choice: 1
Report date (YYYY-MM-DD, Enter = today): 2026-10-20
+-----------+----------+--------------+
| Species   | Total Kg | Revenue (Rs) |
+-----------+----------+--------------+
| Tuna      | 900.0    | 189000.0     |
| Squid     | 300.0    | 84000.0      |
| Seer Fish | 120.0    | 74400.0      |
| Prawn     | 150.0    | 67500.0      |
| Mackerel  | 200.0    | 28000.0      |
| Sardine   | 350.0    | 21000.0      |
+-----------+----------+--------------+
   6 row(s)
  >> Total auction revenue on 2026-10-20 : Rs 463900.0


--- REPORTS ---
1.Daily revenue  2.Species-wise totals  3.Top 5 boats
4.Buyer spending (HAVING)  5.Revenue between dates
6.Export species report to CSV  0.Back
Choice: 2
+-----------+------+----------+----------+----------+----------+
| Species   | Lots | Total Kg | Avg Rate | Max Rate | Revenue  |
+-----------+------+----------+----------+----------+----------+
| Tuna      | 3    | 2800.0   | 205.0    | 210.0    | 573000.0 |
| Squid     | 3    | 870.0    | 283.33   | 290.0    | 246100.0 |
| Seer Fish | 3    | 360.0    | 620.0    | 640.0    | 222400.0 |
| Prawn     | 3    | 380.0    | 460.0    | 470.0    | 174400.0 |
| Mackerel  | 2    | 460.0    | 145.0    | 150.0    | 67000.0  |
| Anchovy   | 2    | 580.0    | 92.5     | 95.0     | 54700.0  |
| Sardine   | 3    | 840.0    | 61.67    | 70.0     | 49300.0  |
| Pomfret   | 1    | 60.0     | 520.0    | 520.0    | 31200.0  |
| Crab      | 1    | 70.0     | 380.0    | 380.0    | 26600.0  |
+-----------+------+----------+----------+----------+----------+
   9 row(s)


--- REPORTS ---
1.Daily revenue  2.Species-wise totals  3.Top 5 boats
4.Buyer spending (HAVING)  5.Revenue between dates
6.Export species report to CSV  0.Back
Choice: 3
+------------+---------+-------+--------+---------------+
| Boat       | Type    | Trips | Kg     | Earnings (Rs) |
+------------+---------+-------+--------+---------------+
| Sea Star   | Trawler | 2     | 2170.0 | 477600.0      |
| Holy Cross | Trawler | 1     | 1350.0 | 292500.0      |
| Our Lady   | Trawler | 1     | 870.0  | 190600.0      |
| St Antony  | Fibre   | 2     | 420.0  | 166400.0      |
| Velankanni | Fibre   | 1     | 200.0  | 115200.0      |
+------------+---------+-------+--------+---------------+
   5 row(s)
  >> TOP BOAT: Sea Star


--- REPORTS ---
1.Daily revenue  2.Species-wise totals  3.Top 5 boats
4.Buyer spending (HAVING)  5.Revenue between dates
6.Export species report to CSV  0.Back
Choice: 4
Show buyers who spent more than Rs: 50000
+-------------------------+------------+------+------------+
| Buyer                   | Type       | Lots | Spent (Rs) |
+-------------------------+------------+------+------------+
| Kanyakumari Sea Exports | Exporter   | 4    | 442400.0   |
| Cape Seafoods           | Exporter   | 3    | 315500.0   |
| Kerala Marine Foods     | Exporter   | 3    | 313200.0   |
| Selvi                   | Vendor     | 3    | 152000.0   |
| Mary Fish Traders       | Wholesaler | 3    | 85800.0    |
| Raja Dry Fish           | Wholesaler | 2    | 69500.0    |
| Ocean Fresh Agency      | Wholesaler | 2    | 60000.0    |
+-------------------------+------------+------+------------+
   7 row(s)


--- REPORTS ---
1.Daily revenue  2.Species-wise totals  3.Top 5 boats
4.Buyer spending (HAVING)  5.Revenue between dates
6.Export species report to CSV  0.Back
Choice: 5
From date (YYYY-MM-DD, Enter = today): 2026-10-20
To date   (YYYY-MM-DD, Enter = today): 2026-10-26
+------------+-------+--------+--------------+
| Date       | Trips | Kg     | Revenue (Rs) |
```

```
+------------+-------+--------+--------------+
| 2026-10-20 | 3     | 2020.0 | 463900.0     |
| 2026-10-21 | 3     | 1630.0 | 414900.0     |
| 2026-10-22 | 3     | 460.0  | 97000.0      |
| 2026-10-23 | 2     | 1270.0 | 212600.0     |
| 2026-10-24 | 2     | 920.0  | 201100.0     |
| 2026-10-25 | 1     | 120.0  | 55200.0      |
+------------+-------+--------+--------------+
   6 row(s)


--- REPORTS ---
1.Daily revenue  2.Species-wise totals  3.Top 5 boats
4.Buyer spending (HAVING)  5.Revenue between dates
6.Export species report to CSV  0.Back
Choice: 6
+-----------+------+----------+----------+----------+----------+
| Species   | Lots | Total Kg | Avg Rate | Max Rate | Revenue  |
+-----------+------+----------+----------+----------+----------+
| Tuna      | 3    | 2800.0   | 205.0    | 210.0    | 573000.0 |
| Squid     | 3    | 870.0    | 283.33   | 290.0    | 246100.0 |
| Seer Fish | 3    | 360.0    | 620.0    | 640.0    | 222400.0 |
| Prawn     | 3    | 380.0    | 460.0    | 470.0    | 174400.0 |
| Mackerel  | 2    | 460.0    | 145.0    | 150.0    | 67000.0  |
| Anchovy   | 2    | 580.0    | 92.5     | 95.0     | 54700.0  |
| Sardine   | 3    | 840.0    | 61.67    | 70.0     | 49300.0  |
| Pomfret   | 1    | 60.0     | 520.0    | 520.0    | 31200.0  |
| Crab      | 1    | 70.0     | 380.0    | 380.0    | 26600.0  |
+-----------+------+----------+----------+----------+----------+
   9 row(s)
  Report saved to species_report.csv
  First 3 lines of the file:
    ['Species', 'Lots', 'Total_Kg', 'Avg_Rate', 'Max_Rate', 'Revenue']
    ['Tuna', '3', '2800.0', '205.0', '210.0', '573000.0']
    ['Squid', '3', '870.0', '283.33', '290.0', '246100.0']


--- REPORTS ---
1.Daily revenue  2.Species-wise totals  3.Top 5 boats
4.Buyer spending (HAVING)  5.Revenue between dates
6.Export species report to CSV  0.Back
Choice: 0


========= MAIN MENU =========
1. Boats & Fishermen
2. Trips & Sea Safety
3. Catch & Auction
4. Buyers
5. Reports
0. Exit
Enter choice: 0


Connection closed. Nanri! (Thank you)
```

8.6 Invalid input at main menu and exit

```
============================================================
   KADAL KANAKKU - Fish Catch, Auction & Sea-Safety Manager
        Manavalakurichi Fish Landing Centre
============================================================


========= MAIN MENU =========
1. Boats & Fishermen
2. Trips & Sea Safety
3. Catch & Auction
4. Buyers
5. Reports
0. Exit
Enter choice: 9
  ! Number must be between 0 and 5
Enter choice: abc
  ! Please enter a whole number.
Enter choice: 0


Connection closed. Nanri! (Thank you)
```

8.7 Error handling – wrong MySQL password

```
============================================================
   KADAL KANAKKU - Fish Catch, Auction & Sea-Safety Manager
        Manavalakurichi Fish Landing Centre
============================================================
Cannot connect to MySQL: Access denied for user 'root'@'localhost' (using password: YES)
```

```
Check that MySQL is running, schema.sql is loaded and
PASSWORD at the top of this file is correct.
```

8.8 Contents of species_report.csv

```
Species,Lots,Total_Kg,Avg_Rate,Max_Rate,Revenue
Tuna,3,2800.0,205.0,210.0,573000.0
Squid,3,870.0,283.33,290.0,246100.0
Seer Fish,3,360.0,620.0,640.0,222400.0
Prawn,3,380.0,460.0,470.0,174400.0
Mackerel,2,460.0,145.0,150.0,67000.0
Anchovy,2,580.0,92.5,95.0,54700.0
Sardine,3,840.0,61.67,70.0,49300.0
Pomfret,1,60.0,520.0,520.0,31200.0
Crab,1,70.0,380.0,380.0,26600.0
```

9. Limitations and Future Scope

Limitations

- Single-user console program; no login or user roles.
- Weather is entered manually, not taken from a live source.
- Auction records only the final winning bid, not every bid.
- IDs of boats, fishermen and buyers are typed manually.

Future Scope

- Graphical (Tkinter) or mobile app interface in Tamil and English.
- Automatic weather alerts from the India Meteorological Department / INCOIS ocean-state forecasts.
- SMS alert to families and the fishermen society when a boat has not returned in time.
- Graphs of fish price trends using matplotlib to help fishermen decide where to sell.
- Link with government welfare schemes and fishing-ban period records.

10. Bibliography

- Computer Science with Python – Textbook for Class XII, Sumita Arora, Dhanpat Rai & Co.
- CBSE Curriculum 2026-27, Computer Science (083) Class XII: https://cbseacademic.nic.in/web_material/CurriculumMain27/SecPart2/Computer_Science_SecP2_2026-27.pdf
- CBSE Sample Question Paper 2026-27 (Computer Science): https://cbseacademic.nic.in
- MySQL 8.0 Reference Manual: https://dev.mysql.com/doc/
- MySQL Connector/Python Developer Guide: https://dev.mysql.com/doc/connector-python/en/
- Python 3 documentation (csv module): https://docs.python.org/3/library/csv.html
- Discussions with local fishermen and merchants at Manavalakurichi beach.